NumPy Full Course Notes — Beginner to Advanced

Python | Bangla Explanation | Practical Examples

---

1. Introduction to NumPy

What is NumPy?

NumPy (Numerical Python) হলো Python-এর একটি powerful library, যা numerical calculation, scientific computing এবং large datasets নিয়ে কাজ করার জন্য ব্যবহার করা হয়।

NumPy-এর প্রধান object হলো `ndarray` (N-dimensional array)।

Why use NumPy?

- Fast numerical calculations
- Multi-dimensional arrays
- Mathematical operations
- Linear algebra
- Statistical operations
- Random number generation
- Data analysis and scientific computing
- Memory-efficient calculations

Install NumPy

```
pip install numpy
```

Import NumPy

```
import numpy as np
```

`np` হলো NumPy-এর standard alias।

---

2. Python List vs NumPy Array

Python List

```
numbers = [1, 2, 3, 4, 5]

result = [x * 2 for x in numbers]

print(result)
```

Output:

```
[2, 4, 6, 8, 10]
```

NumPy Array

```
import numpy as np

numbers = np.array([1, 2, 3, 4, 5])

result = numbers * 2

print(result)
```

Output:

```
[ 2  4  6  8 10]
```

Main Differences

Python List	NumPy Array
General-purpose collection	Numerical computing
Can contain mixed types	Usually one common dtype
Slower numerical operations	Faster numerical operations
No built-in vectorized math	Supports vectorized math
More flexible	More memory efficient for many numerical workloads

---

3. Creating NumPy Arrays

3.1 np.array()

```
import numpy as np

arr = np.array([10, 20, 30, 40])

print(arr)
```

Output:

```
[10 20 30 40]
```

3.2 One-Dimensional Array

```
arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

3.3 Two-Dimensional Array

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(arr)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

3.4 Three-Dimensional Array

```
arr = np.array([
    [
        [1, 2],
        [3, 4]
    ],
    [
        [5, 6],
        [7, 8]
    ]
])

print(arr)
```

3.5 np.zeros()

Creates an array filled with zeros.

```
arr = np.zeros(5)

print(arr)
```

Output:

```
[0. 0. 0. 0. 0.]
```

2D:

```
arr = np.zeros((3, 4))

print(arr)
```

3.6 np.ones()

```
arr = np.ones((2, 3))

print(arr)
```

3.7 np.full()

Creates an array filled with a specific value.

```
arr = np.full((2, 3), 7)

print(arr)
```

Output:

```
[[7 7 7]
 [7 7 7]]
```

3.8 np.empty()

Creates an uninitialized array. Its values are not guaranteed to be zero.

```
arr = np.empty((2, 3))

print(arr)
```

Use it when you will overwrite every element before reading it.

3.9 np.arange()

Creates values using a step.

```
arr = np.arange(1, 10)

print(arr)
```

Output:

```
[1 2 3 4 5 6 7 8 9]
```

With step:

```
arr = np.arange(0, 20, 2)

print(arr)
```

Output:

```
[ 0  2  4  6  8 10 12 14 16 18]
```

Syntax:

```
np.arange(start, stop, step)
```

`stop` value included হয় না।

3.10 np.linspace()

Creates a fixed number of evenly spaced values.

```
arr = np.linspace(0, 10, 5)

print(arr)
```

Output:

```
[ 0.   2.5  5.   7.5 10. ]
```

Difference:

- `arange()` নির্দিষ্ট step অনুযায়ী values তৈরি করে।
- `linspace()` নির্দিষ্ট সংখ্যক values তৈরি করে।

---

4. NumPy Array Attributes

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(arr.ndim)
print(arr.shape)
print(arr.size)
print(arr.dtype)
print(arr.itemsize)
print(arr.nbytes)
```

Important Attributes

Attribute	Meaning
ndim	Number of dimensions
shape	Array-এর size in each dimension
size	Total number of elements
dtype	Data type
itemsize	Bytes per element
nbytes	Total array element memory

Example:

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr.ndim)      # 2
print(arr.shape)     # (2, 3)
```

```
print(arr.size)      # 6
print(arr.dtype)     # int64 or platform-dependent integer type
```

---

5. NumPy Data Types (dtype)

NumPy array সাধারণত একটি common data type ব্যবহার করে।

```
arr = np.array([1, 2, 3])

print(arr.dtype)
```

Common Data Types

Type	Description
int8	8-bit integer
int16	16-bit integer
int32	32-bit integer
int64	64-bit integer
float32	32-bit float
float64	64-bit float
bool_	Boolean
complex64	Complex number

Specify dtype

```
arr = np.array([1, 2, 3], dtype=np.float64)

print(arr)
print(arr.dtype)
```

Type Conversion

```
arr = np.array([1.2, 2.8, 3.5])

new_arr = arr.astype(np.int32)

print(new_arr)
```

Output:

```
[1 2 3]
```

`astype()` নতুন array তৈরি করে।

---

6. Indexing and Slicing

6.1 One-Dimensional Indexing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])    # 10
print(arr[2])    # 30
print(arr[-1])   # 50
```

6.2 Slicing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[1:4])
```

Output:

```
[20 30 40]
```

Syntax:

```
arr[start:stop:step]
```

6.3 Two-Dimensional Indexing

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print(arr[0, 0])   # 10
print(arr[1, 2])   # 60
```

6.4 2D Slicing

```
print(arr[0, :])
print(arr[:, 1])
```

Meaning:

- `arr[0, :]` → প্রথম row
- `arr[:, 1]` → দ্বিতীয় column

6.5 Negative Indexing

```
arr = np.array([10, 20, 30, 40])

print(arr[-1])
print(arr[-2])
```

6.6 Fancy Indexing

একাধিক নির্দিষ্ট index থেকে element নেওয়া যায়।

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[[0, 2, 4]])
```

Output:

```
[10 30 50]
```

---

7. Modifying Arrays

```
arr = np.array([10, 20, 30, 40])

arr[0] = 100

print(arr)
```

2D:

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

arr[0, 1] = 99

print(arr)
```

Modify a complete row:

```
arr[0] = [10, 20, 30]
```

Modify a complete column:

```
arr[:, 1] = [100, 200]
```

---

8. Reshaping Arrays

Reshape দিয়ে array-এর dimensions পরিবর্তন করা যায়।

```
arr = np.arange(1, 13)

new_arr = arr.reshape(3, 4)

print(new_arr)
```

Output:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

Using -1

NumPy একটি dimension নিজে calculate করে নিতে পারে।

```
arr = np.arange(12)

new_arr = arr.reshape(3, -1)

print(new_arr)
```

Flatten

Multi-dimensional array-কে 1D array বানায়।

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

flat = arr.flatten()

print(flat)
```

`flatten()` একটি copy তৈরি করে।

Ravel

```
flat = arr.ravel()
```

`ravel()` সম্ভব হলে view return করে।

Transpose

Rows এবং columns পরিবর্তন করে।

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(arr.T)
```

---

9. Joining Arrays

9.1 np.concatenate()

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = np.concatenate((a, b))

print(result)
```

2D:

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

result = np.concatenate((a, b), axis=0)

print(result)
```

9.2 np.vstack()

Vertical stacking:

```
a = np.array([1, 2])
b = np.array([3, 4])

print(np.vstack((a, b)))
```

9.3 np.hstack()

Horizontal stacking:

```
print(np.hstack((a, b)))
```

9.4 np.stack()

নতুন axis তৈরি করে arrays combine করে।

```
a = np.array([1, 2])
b = np.array([3, 4])

print(np.stack((a, b), axis=0))
```

---

10. Splitting Arrays

np.split()

```
arr = np.array([1, 2, 3, 4, 5, 6])

result = np.split(arr, 3)

print(result)
```

np.array_split()

Unequal-sized sections-ও তৈরি করতে পারে।

```
arr = np.array([1, 2, 3, 4, 5])

result = np.array_split(arr, 3)

print(result)
```

Horizontal and Vertical Split

```
arr = np.arange(16).reshape(4, 4)

print(np.hsplit(arr, 2))
print(np.vsplit(arr, 2))
```

---

11. Mathematical Operations

NumPy array-এর ওপর সরাসরি mathematical operations করা যায়।

```
arr = np.array([10, 20, 30, 40])

print(arr + 5)
```

```
print(arr - 5)
print(arr * 2)
print(arr / 2)
print(arr ** 2)
print(arr % 3)
```

Array-to-Array Operations

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

এগুলো element-wise operations।

Universal Functions (ufuncs)

```
arr = np.array([1, 4, 9, 16])

print(np.sqrt(arr))
print(np.square(arr))
print(np.exp(arr))
print(np.log(arr))
print(np.abs(arr))
```

Trigonometric:

```
angles = np.array([0, 30, 45, 60, 90])

radians = np.deg2rad(angles)

print(np.sin(radians))
print(np.cos(radians))
print(np.tan(radians))
```

---

12. Aggregation Functions

Aggregation মানে অনেকগুলো value থেকে একটি summary result বের করা।

```
arr = np.array([10, 20, 30, 40, 50])

print(np.sum(arr))
print(np.min(arr))
print(np.max(arr))
print(np.mean(arr))
print(np.median(arr))
print(np.std(arr))
print(np.var(arr))
print(np.prod(arr))
```

Important Functions

Function	Purpose
sum()	Total
min()	Minimum
max()	Maximum
mean()	Average
median()	Middle value
std()	Standard deviation
var()	Variance
prod()	Product
argmin()	Index of minimum
argmax()	Index of maximum
cumsum()	Cumulative sum
cumprod()	Cumulative product

Example:

```
arr = np.array([10, 20, 30, 40])

print(np.argmax(arr))  # 3
print(np.argmin(arr))  # 0

print(np.cumsum(arr))
```

---

13. Understanding axis

NumPy-এর সবচেয়ে গুরুত্বপূর্ণ concepts-এর একটি হলো `axis`।
2D array:

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

axis=0

Column অনুযায়ী operation হয়।

```
print(np.sum(arr, axis=0))
```

Output:

```
[50 70 90]
```

axis=1

Row অনুযায়ী operation হয়।

```
print(np.sum(arr, axis=1))
```

Output:

```
[ 60 150]
```

Remember

- `axis=0` → rows collapse হয়, column-wise result পাওয়া যায়।
- `axis=1` → columns collapse হয়, row-wise result পাওয়া যায়।

---

14. Comparison Operations

```
arr = np.array([10, 20, 30, 40, 50])

print(arr > 25)
print(arr < 30)
print(arr == 20)
print(arr != 40)
```

Output:

```
[False False  True  True  True]
```

---

15. Boolean Indexing

Condition অনুযায়ী values filter করা যায়।

```
arr = np.array([10, 20, 30, 40, 50])

result = arr[arr > 25]

print(result)
```

Output:

```
[30 40 50]
```

Multiple conditions:

```
result = arr[(arr > 20) & (arr < 50)]

print(result)
```

OR condition:

```
result = arr[(arr < 20) | (arr > 40)]
```

Important:

- AND → `&`
- OR → `|`
- NOT → `~`

প্রতিটি condition-কে parentheses-এর মধ্যে রাখবে।

---

16. np.where()

Condition অনুযায়ী value select বা replace করা যায়।

```
arr = np.array([10, 20, 30, 40, 50])

result = np.where(arr > 25, 1, 0)

print(result)
```

Output:

```
[0 0 1 1 1]
```

Syntax:

```
np.where(condition, value_if_true, value_if_false)
```

Practical example:

```
marks = np.array([30, 70, 45, 90, 20])

status = np.where(marks >= 40, "Pass", "Fail")

print(status)
```

---

17. Sorting Arrays

np.sort()

```
arr = np.array([40, 10, 50, 20, 30])

print(np.sort(arr))
```

Original array unchanged থাকে।

Descending Order

```
print(np.sort(arr)[::-1])
```

np.argsort()

Sorted order অনুযায়ী original indices return করে।

```
arr = np.array([40, 10, 30])

print(np.argsort(arr))
```

2D Sorting

```
arr = np.array([
    [3, 2, 1],
```

```
    [6, 5, 4]
])

print(np.sort(arr, axis=0))
print(np.sort(arr, axis=1))
```

---

18. Searching Arrays

np.argmax()

```
arr = np.array([10, 50, 20, 40])

print(np.argmax(arr))
```

np.argwhere()

Condition-এর matching indices return করে।

```
arr = np.array([10, 20, 30, 40])

print(np.argwhere(arr > 20))
```

np.nonzero()

Non-zero element-এর indices return করে।

```
arr = np.array([0, 10, 0, 20])

print(np.nonzero(arr))
```

np.searchsorted()

Sorted array-তে insertion position খুঁজে দেয়।

```
arr = np.array([10, 20, 30, 40])

print(np.searchsorted(arr, 25))
```

Output:

```
2
```

---

19. Broadcasting

Broadcasting হলো compatible shapes-এর arrays-এর মধ্যে operation করার নিয়ম।

Example 1

```
a = np.array([1, 2, 3])

print(a + 10)
```

Output:

```
[11 12 13]
```

Example 2

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

b = np.array([10, 20, 30])

print(a + b)
```

Output:

```
[[11 22 33]
 [14 25 36]]
```

Broadcasting Rules

দুটি shape ডান দিক থেকে তুলনা করা হয়।

Dimensions compatible যদি:

1. দুই dimension সমান হয়, অথবা
2. একটির dimension `1` হয়।

কোনো dimension-এ এই নিয়ম না মিললে broadcasting error হবে।

Example:

```
a = np.ones((3, 1))
b = np.ones((1, 4))

result = a + b

print(result.shape)
```

Output:

```
(3, 4)
```

---

20. Copy vs View

এটি খুব গুরুত্বপূর্ণ concept।

View

View original data share করে।

```
arr = np.array([1, 2, 3, 4])

view_arr = arr.view()

view_arr[0] = 100

print(arr)
```

Original array-ও পরিবর্তিত হবে।

Copy

Copy আলাদা data তৈরি করে।

```
arr = np.array([1, 2, 3, 4])
```

```
copy_arr = arr.copy()

copy_arr[0] = 100

print(arr)
```

Original array পরিবর্তন হবে না।

Difference

Copy	View
Separate data	Shared data
Original unaffected	Changes may affect original
Uses additional data memory	Usually avoids data copy

Advanced note: Slicing সাধারণত view দেয়, কিন্তু integer-array বা Boolean indexing সাধারণত copy দেয়।
`reshape()` -ও সম্ভব হলে view দেয়, না হলে copy তৈরি করতে পারে।

---

21. Random Module

```
import numpy as np

rng = np.random.default_rng(42)
```

`default_rng()` হলো recommended random generator API।

Random Float

```
print(rng.random(5))
```

Random Integer

```
print(rng.integers(1, 100, size=5))
```

Random 2D Array

```
print(rng.random((3, 3)))
```

Normal Distribution

```
print(rng.normal(0, 1, size=5))
```

Choice

```
arr = np.array([10, 20, 30, 40])

print(rng.choice(arr, size=2))
```

Shuffle

```
arr = np.array([1, 2, 3, 4, 5])

rng.shuffle(arr)

print(arr)
```

Permutation

```
arr = np.array([1, 2, 3, 4])

print(rng.permutation(arr))
```

`shuffle()` input array পরিবর্তন করে; `permutation()` নতুন permuted result দেয়।

---

22. Matrix Operations

Matrix Creation

```
a = np.array([
    [1, 2],
    [3, 4]
])

b = np.array([
    [5, 6],
    [7, 8]
])
```

Matrix Addition

```
print(a + b)
```

Matrix Multiplication

Element-wise:

```
print(a * b)
```

Matrix multiplication:

```
print(a @ b)
```

অথবা:

```
print(np.matmul(a, b))
```

Dot Product

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
```

```
print(np.dot(x, y))
```

Matrix Transpose

```
print(a.T)
```

Determinant

```
print(np.linalg.det(a))
```

Inverse

```
print(np.linalg.inv(a))
```

Eigenvalues and Eigenvectors

```
values, vectors = np.linalg.eig(a)

print(values)
print(vectors)
```

Solve Linear Equations

```
A = np.array([
    [2, 1],
    [1, 3]
])

B = np.array([8, 13])

solution = np.linalg.solve(A, B)

print(solution)
```

---

23. Unique and Set Operations

np.unique()

```
arr = np.array([1, 2, 2, 3, 3, 3])

print(np.unique(arr))
```

Return Counts

```
values, counts = np.unique(arr, return_counts=True)

print(values)
print(counts)
```

Union

```
a = np.array([1, 2, 3])
b = np.array([3, 4, 5])

print(np.union1d(a, b))
```

Intersection

```
print(np.intersect1d(a, b))
```

Difference

```
print(np.setdiff1d(a, b))
```

---

24. Handling NaN and Infinity

NaN মানে Not a Number।

```
arr = np.array([10, 20, np.nan, 40])

print(np.isnan(arr))
```

Remove NaN

```
clean = arr[~np.isnan(arr)]

print(clean)
```

nanmean()

```
print(np.nanmean(arr))
```

nanmax()

```
print(np.nanmax(arr))
```

Infinity

```
arr = np.array([10, np.inf, -np.inf])

print(np.isinf(arr))
```

Check Finite Values

```
print(np.isfinite(arr))
```

---

25. Adding and Removing Elements

np.append()

```
arr = np.array([1, 2, 3])

new_arr = np.append(arr, 4)

print(new_arr)
```

np.insert()

```
arr = np.array([10, 20, 30])

new_arr = np.insert(arr, 1, 99)
```

```
print(new_arr)
```

np.delete()

```
arr = np.array([10, 20, 30, 40])

new_arr = np.delete(arr, 1)

print(new_arr)
```

Important: These operations create new arrays. They do not modify the original array in place.

---

26. Saving and Loading Arrays

Save .npy

```
arr = np.array([10, 20, 30])

np.save("data.npy", arr)
```

Load:

```
data = np.load("data.npy")

print(data)
```

Save Multiple Arrays

```
np.savez("data.npz", numbers=arr, doubled=arr * 2)
```

Load:

```
data = np.load("data.npz")

print(data["numbers"])
print(data["doubled"])
```

Save CSV / Text

```
np.savetxt("data.csv", arr, delimiter=",")
```

Load:

```
data = np.loadtxt("data.csv", delimiter=",")

print(data)
```

For untrusted `.npz` files, use `allow_pickle=False` when loading where possible.

---

27. Advanced Array Manipulation

np.expand_dims()

নতুন dimension যোগ করে।

```
arr = np.array([1, 2, 3])
```

```
result = np.expand_dims(arr, axis=0)

print(result.shape)
```

Output:

```
(1, 3)
```

np.squeeze()

Size 1-এর dimensions remove করে।

```
arr = np.array([[[1, 2, 3]]])

result = np.squeeze(arr)

print(result.shape)
```

np.swapaxes()

দুটি axes swap করে।

```
arr = np.zeros((2, 3, 4))

result = np.swapaxes(arr, 0, 2)

print(result.shape)
```

np.moveaxis()

একটি axis অন্য position-এ move করে।

```
arr = np.zeros((2, 3, 4))

result = np.moveaxis(arr, 0, -1)

print(result.shape)
```

np.repeat()

```
arr = np.array([1, 2, 3])

print(np.repeat(arr, 2))
```

np.tile()

```
arr = np.array([1, 2, 3])

print(np.tile(arr, 3))
```

---

28. Advanced Indexing with np.ix_

Multiple rows এবং columns একসঙ্গে select করা যায়।

```
arr = np.arange(16).reshape(4, 4)

rows = [0, 2]
cols = [1, 3]
```

```
result = arr[np.ix_(rows, cols)]

print(result)
```

`np.ix_()` Cartesian selection তৈরি করে।

---

29. Vectorization

Vectorization হলো Python loop ছাড়াই array-এর ওপর operation করা।

Without Vectorization

```
numbers = [1, 2, 3, 4, 5]

result = []

for number in numbers:
    result.append(number * 2)

print(result)
```

With Vectorization

```
arr = np.array([1, 2, 3, 4, 5])

result = arr * 2

print(result)
```

Benefits:

- Less code
- Faster numerical calculations in many cases
- Better readability
- Uses optimized low-level operations

---

30. np.vectorize()

Python function-কে array-এর element-এ apply করার সুবিধা দেয়।

```
def square(x):
    return x ** 2

vectorized_square = np.vectorize(square)

arr = np.array([1, 2, 3, 4])

print(vectorized_square(arr))
```

Important: `np.vectorize()` মূলত convenience tool। এটি সাধারণত performance vectorization নয় এবং built-in NumPy operations-এর মতো speed guarantee করে না।

---

31. Memory Layout and Contiguous Arrays

NumPy arrays memory-তে কীভাবে data রাখে তা performance-এ প্রভাব ফেলতে পারে।

```
arr = np.arange(12).reshape(3, 4)

print(arr.flags)
print(arr.strides)
print(arr.flags["C_CONTIGUOUS"])
print(arr.flags["F_CONTIGUOUS"])
```

C Order

Row-major layout।

```
arr = np.array([[1, 2], [3, 4]], order="C")
```

Fortran Order

Column-major layout।

```
arr = np.array([[1, 2], [3, 4]], order="F")
```

Make Contiguous

```
arr = np.ascontiguousarray(arr)
```

---

32. Structured Arrays

একটি NumPy array-তে named fields রাখা যায়।

```
data = np.array(
    [
        (1, "Faruk", 25.5),
        (2, "Rahim", 30.5)
    ],
    dtype=[
        ("id", "i4"),
        ("name", "U20"),
        ("salary", "f8")
    ]
)

print(data["name"])
print(data["salary"])
```

Structured arrays specialized data representation-এর জন্য কাজে লাগে।

---

33. Masked Arrays

Missing বা invalid values mask করার জন্য ব্যবহার করা যায়।

```
arr = np.array([10, 20, -999, 40])

masked = np.ma.masked_equal(arr, -999)

print(masked)
print(masked.mean())
```

---

34. Polynomial Operations

```
coefficients = np.array([1, 2, 3])

# x^2 + 2x + 3
print(np.polyval(coefficients, 2))
```

Polynomial fitting:

```
x = np.array([1, 2, 3, 4])
y = np.array([2, 4, 6, 8])

coefficients = np.polyfit(x, y, 1)

print(coefficients)
```

For new code, also explore the newer `numpy.polynomial` API.

---

35. Statistics and Percentiles

```
data = np.array([10, 20, 30, 40, 50])

print(np.percentile(data, 25))
print(np.percentile(data, 50))
print(np.percentile(data, 75))
```

Quantile:

```
print(np.quantile(data, 0.5))
```

Correlation:

```
x = np.array([1, 2, 3, 4])
y = np.array([2, 4, 6, 8])

print(np.corrcoef(x, y))
```

Histogram:

```
data = np.array([1, 2, 2, 3, 3, 3, 4, 5])

hist, bins = np.histogram(data, bins=4)

print(hist)
print(bins)
```

---

36. Working with Dates

NumPy supports date and time arrays.

```
dates = np.array(
    ["2026-01-01", "2026-01-02", "2026-01-03"],
    dtype="datetime64[D]"
)

print(dates)
```

Date difference:

```
start = np.datetime64("2026-01-01")
end = np.datetime64("2026-01-10")

print(end - start)
```

Date range:

```
dates = np.arange(
    np.datetime64("2026-01-01"),
    np.datetime64("2026-01-10"),
    dtype="datetime64[D]"
)

print(dates)
```

---

37. Performance Optimization

1. Prefer Vectorized Operations

```
arr = np.arange(1_000_000)

result = arr * 2
```

2. Choose Appropriate dtype

```
arr = np.array([1, 2, 3], dtype=np.int32)
```

3. Avoid Unnecessary Copies

```
view = arr.view()
```

Use only when shared-memory behavior is safe.

4. Preallocate Arrays

```
result = np.empty(5)

for i in range(5):
    result[i] = i * 2
```

For larger numerical workflows, consider vectorizing the calculation instead of looping.

5. Use In-place Operations When Appropriate

```
arr = np.array([1, 2, 3])

arr *= 2
```

In-place operations can reduce allocations, but may mutate data that other variables share.

---

38. NumPy with Pandas

NumPy এবং Pandas একসঙ্গে Data Science-এ ব্যবহার করা হয়।

```
import numpy as np
import pandas as pd
```

```
arr = np.array([10, 20, 30, 40])

df = pd.DataFrame({
    "marks": arr
})

print(df)
```

Pandas to NumPy:

```
arr = df["marks"].to_numpy()

print(arr)
```

---

39. Real-World Practical Examples

Project 1: Student Marks Analysis

```
import numpy as np

marks = np.array([80, 75, 90, 65, 85])

print("Total:", np.sum(marks))
print("Average:", np.mean(marks))
print("Highest:", np.max(marks))
print("Lowest:", np.min(marks))

passed = marks[marks >= 40]

print("Passed:", passed)
print("Pass Count:", len(passed))
```

Project 2: Sales Analysis

```
import numpy as np

sales = np.array([
    [100, 200, 300],
    [150, 250, 350],
    [200, 300, 400]
])

daily_sales = np.sum(sales, axis=1)
product_sales = np.sum(sales, axis=0)

print("Daily Sales:", daily_sales)
print("Product Sales:", product_sales)
print("Total Sales:", np.sum(sales))
```

Project 3: Image Processing Basics

An image can be represented as a NumPy array.

```
image = np.zeros((100, 100, 3), dtype=np.uint8)
```

```
# Set one pixel to white
image[50, 50] = [255, 255, 255]

print(image.shape)
```

Project 4: Normalize Data

```
data = np.array([10, 20, 30, 40, 50])

normalized = (data - np.min(data)) / (
    np.max(data) - np.min(data)
)

print(normalized)
```

This is Min-Max normalization. If all values are equal, the denominator is zero, so handle that case in production code.

Project 5: Standardization

```
data = np.array([10, 20, 30, 40, 50])

standardized = (
    data - np.mean(data)
) / np.std(data)

print(standardized)
```

---

40. NumPy Interview Questions

Beginner Questions

1. What is NumPy?

NumPy is a Python library for numerical computing. Its core object is the N-dimensional array.

2. What is ndarray?

`ndarray` হলো NumPy-এর multi-dimensional array object।

3. How do you create an array?

```
arr = np.array([1, 2, 3])
```

4. What is the difference between shape and size?

- `shape` → প্রতিটি dimension-এর size।
- `size` → total number of elements।

5. What is dtype?

Array-এর element data type।

6. What is the difference between arange and linspace?

- `arange()` → step অনুযায়ী values।
- `linspace()` → নির্দিষ্ট সংখ্যক evenly spaced values।

Intermediate Questions

7. What is broadcasting?

Compatible shapes-এর arrays-এর মধ্যে element-wise operations করার নিয়ম।

8. What is vectorization?

Python loop ছাড়াই array operations করা।

9. What is the difference between copy and view?

- Copy → independent data।
- View → shared underlying data।

10. What is axis=0?

2D array-তে column-wise reduction।

11. What is axis=1?

2D array-তে row-wise reduction।

12. What is Boolean indexing?

Boolean condition ব্যবহার করে elements filter করা।

Advanced Questions

13. What is memory contiguity?

Array-এর data memory-তে contiguous বা নির্দিষ্ট layout-এ রাখা হয়েছে কি না।

14. What is stride?

একটি dimension বরাবর পরবর্তী element-এ যেতে memory-তে কত bytes অতিক্রম করতে হয়।

15. What is a ufunc?

NumPy-এর optimized element-wise universal function।

16. What is np.einsum()?

Einstein summation notation ব্যবহার করে array operations প্রকাশ করার উপায়।
Example:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.einsum("i,i->", a, b))
```

17. What is np.matmul()?

Matrix multiplication operation।

18. What is the difference between * and @?

- `*` → element-wise multiplication।
- `@` → matrix multiplication।

19. What is np.linalg.solve()?

Linear equations-এর system solve করে।

20. Does np.vectorize() make a Python function fast?

না। এটি convenience wrapper; সাধারণত performance optimization নয়।

---

41. Practice Problems

Beginner Level

1. Create a NumPy array containing numbers from 1 to 10.
2. Create a 3×3 zero matrix.
3. Create a 4×4 identity matrix.
4. Find the maximum and minimum values.
5. Calculate the mean of an array.

6. Reverse an array.
7. Extract even numbers.
8. Convert a 1D array into a 2D array.
9. Change the dtype to float32.
10. Find the sum of all elements.

Intermediate Level

1. Find all numbers greater than 50.
2. Replace values below 40 with zero.
3. Sort a 2D array by columns.
4. Find duplicate values.
5. Count the frequency of each unique value.
6. Combine two arrays vertically.
7. Split an array into three parts.
8. Normalize an array.
9. Find row-wise and column-wise sums.
10. Find the indices of maximum values.

Advanced Level

1. Implement Min-Max normalization.
2. Implement Z-score standardization.
3. Solve a system of linear equations.
4. Calculate covariance and correlation.
5. Create a random dataset with a fixed seed.
6. Use broadcasting to calculate pairwise differences.
7. Use `np.einsum()` for matrix multiplication.
8. Handle missing values using NaN functions.
9. Compare memory usage for different dtypes.
10. Analyze a multidimensional dataset using axis operations.

---

42. Complete Learning Roadmap

Level 1 — Beginner

- NumPy introduction
- Installation and import
- Array creation
- 1D, 2D, 3D arrays
- Array attributes
- Data types
- Indexing and slicing
- Reshape
- Basic mathematical operations
- Aggregation functions

Level 2 — Intermediate

- Broadcasting
- Boolean indexing

- Fancy indexing
- Sorting
- Searching
- Joining and splitting
- Copy vs view
- Random module
- Unique and set operations
- NaN handling

Level 3 — Advanced

- Matrix operations
- Linear algebra
- Vectorization
- Advanced indexing
- Memory layout
- Strides
- Structured arrays
- Masked arrays
- Statistics
- Date and time operations

Level 4 — Expert

- Performance optimization
- Memory-efficient computation
- Broadcasting internals
- Ufuncs
- `np.einsum()`
- Numerical stability
- Array interoperability
- Working with large datasets
- NumPy with Pandas and SciPy

---

43. Recommended Study Order

1. Understand Python lists first.
2. Learn 1D and 2D arrays.
3. Practice indexing and slicing.
4. Learn array attributes and dtype.
5. Master reshape and axis.
6. Practice mathematical operations.
7. Understand broadcasting.
8. Learn Boolean and fancy indexing.
9. Master copy vs view.
10. Learn random and statistics.
11. Study matrix and linear algebra.
12. Practice performance optimization.
13. Build practical projects.

Final Advice

NumPy ভালোভাবে শিখতে শুধু notes পড়লে হবে না। প্রতিটি topic নিজে code করে practice করতে হবে।

Recommended routine:

- 30 minutes: Learn concepts.
- 30 minutes: Write code.
- 30 minutes: Solve problems.
- 30 minutes: Build small projects.

Goal: NumPy ব্যবহার করে numerical data efficiently create, manipulate, analyze এবং transform করতে পারা।